# request_log.py - Remember What Happened



## What this file does (big picture)

request_log.py is a learning-friendly diary for each support request.

After every question, main.py writes one line with:
- timestamp
- question
- classification (lane), if we got that far
- action taken (order lookup, knowledge reply, guardrail block, ...)
- outcome (answered / escalated / blocked)
- optional notes

Format: JSONL (one JSON object per line) in data/logs/support_requests.jsonl
Log files are gitignored so practice questions do not get committed.

## New words you will see in this file

| | |
|---|---|
| **observability** | Making the system's behavior visible after the fact. |
| **JSONL** | JSON Lines: one complete JSON record per line; easy to append. |
| **append-only** | We add lines; we do not rewrite history each run. |
| **outcome vs action** | Outcome = how it ended; action = what we did to get there. |

### Flow while the program runs

main finishes a path (answer / block / escalate) ->
log_request(...) opens support_requests.jsonl and appends one line.

# request_log.py - Remember What Happened



**1-12**

```
"""Simple request log..."""
```

**Plain English:**
Module docstring: learning log, not full compliance audit.

**14-18**

```
imports: json, datetime, Path
```

**Plain English:**
JSON for the record shape; UTC timestamps; Path for the log file location.

**20-22**

```
LOG_DIR / LOG_PATH
```

**Plain English:**
data/logs/support_requests.jsonl under the project root.

**24-36**

```
OUTCOME_* and ACTION_* constants
```

**Plain English:**
Shared vocabulary so main.py does not scatter magic strings.

**39-66**

```
def log_request(...)
```

**Plain English:**
Build a dict, ensure the folder exists, append one JSON line, return the path.